SQL MINI LAB PROJECT

Food Ordering & Employee Management System

Course Outcome:

Understand and use database queries and web technologies to build interactive applications.

Submitted By: Rohit

1. Database Creation

```
CREATE DATABASE mini_lab;
USE mini_lab;
```

2. Table Structures

```
CREATE TABLE users (
 id INT PRIMARY KEY AUTO_INCREMENT,
 username VARCHAR(50),
 email VARCHAR(100),
 mobile VARCHAR(15),
 phone VARCHAR(15),
 created_at DATETIME
);

CREATE TABLE orders (
 order_id INT PRIMARY KEY AUTO_INCREMENT,
 user_id INT,
 total_amount DECIMAL(10,2) DEFAULT 0,
 status VARCHAR(20),
 created_at DATETIME
);

CREATE TABLE order_items (
 item_id INT PRIMARY KEY AUTO_INCREMENT,
 order_id INT,
 dish_name VARCHAR(100),
 price DECIMAL(10,2)
);

CREATE TABLE employee (
 id INT PRIMARY KEY AUTO_INCREMENT,
 name VARCHAR(50),
 salary DECIMAL(10,2),
 department VARCHAR(50),
 joining_date DATE
);

CREATE TABLE employee_audit (
 audit_id INT PRIMARY KEY AUTO_INCREMENT,
 emp_id INT,
 old_salary DECIMAL(10,2),
 updated_at DATETIME
);
```

3. SQL Triggers

```
-- Auto set created_at
CREATE TRIGGER set_user_created_at
BEFORE INSERT ON users
FOR EACH ROW
SET NEW.created_at = NOW();

-- Prevent Negative Salary
DELIMITER //
CREATE TRIGGER check_salary
BEFORE INSERT ON employee
FOR EACH ROW
BEGIN
 IF NEW.salary < 0 THEN
  SIGNAL SQLSTATE '45000'
  SET MESSAGE_TEXT = 'Salary cannot be negative';
 END IF;
END //
DELIMITER ;

-- Update Order Total Automatically
DELIMITER //
CREATE TRIGGER update_order_total
AFTER INSERT ON order_items
FOR EACH ROW
BEGIN
 UPDATE orders
 SET total_amount = total_amount + NEW.price
 WHERE order_id = NEW.order_id;
END //
DELIMITER ;

-- Maintain Employee Audit Log
DELIMITER //
CREATE TRIGGER employee_update_audit
AFTER UPDATE ON employee
FOR EACH ROW
BEGIN
 INSERT INTO employee_audit(emp_id, old_salary, updated_at)
 VALUES(OLD.id, OLD.salary, NOW());
END //
DELIMITER ;
```

4. Indexes

```
CREATE INDEX idx_email ON users(email);
CREATE UNIQUE INDEX idx_username ON users(username);
CREATE INDEX idx_status_created ON orders(status, created_at);
```

5. User Defined Functions

```
-- Net Salary after 10% tax
DELIMITER //
CREATE FUNCTION net_salary(salary DECIMAL(10,2))
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
 RETURN salary - (salary * 0.10);
END //
DELIMITER ;

-- Late Fee Calculator
DELIMITER //
CREATE FUNCTION late_fee(days_late INT)
RETURNS INT
DETERMINISTIC
BEGIN
 DECLARE fee INT;
 SET fee = days_late * 50;
 IF fee > 1000 THEN
  SET fee = 1000;
 END IF;
 RETURN fee;
END //
DELIMITER ;
```

6. Conclusion

This project demonstrates the use of SQL Triggers, Indexes, Built-in Functions, Date Handling, and User Defined Functions to build an interactive database-driven application. It improves data integrity, performance, and automation within the system.